

SIMATS ENGINEERING
(Saveetha School of Engineering)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 2
Code Review & Repository Evaluation Report

Problem Statement 33
AI-Powered Smart Municipal Asset Management and Maintenance Platform

Field	Details
Course Code	CSA10 — Software Engineering
CO Assessed	CO3 — Build full-stack applications with an emphasis on containerization, version control, and collaborative development
Assessment	Assessment Tool 2 — Code Review & Repository Evaluation
Weightage	20% Total Marks: 25
Student Name	Jaswanth.S
Branch	AIML — Computer Science and Engineering
Project Title	AI-Powered Smart Municipal Asset Management and Maintenance Platform
Repository	github.com/jaswanths/smart-muni-asset-platform

1. Problem Overview

1.1 Assigned Problem Statement

Municipal corporations are responsible for maintaining a wide range of public assets, including roads, parks, streetlights, drainage systems, public buildings, and water infrastructure. In most municipal bodies, this maintenance is still managed through fragmented, paper-based, or spreadsheet-driven systems. As a result, assets deteriorate due to delayed inspections, repair schedules are decided reactively rather than proactively, and there is no centralized view of infrastructure health across the city. The assigned problem statement calls for an AI-powered municipal asset management platform that digitizes the asset inventory, continuously monitors asset condition through IoT sensors, predicts maintenance needs before failures occur, prioritizes repair schedules based on urgency and impact, and produces infrastructure performance reports for decision-makers.

1.2 Summary of the Implemented Solution

The implemented solution is a full-stack web platform, Smart Municipal Asset Management and Maintenance Platform (internally referred to as SM-AMP), that gives municipal engineers a single dashboard to register assets, track their condition over time, and plan maintenance work. The system is composed of four cooperating parts: a React.js single-page frontend for dashboards and GIS-based asset maps, a Node.js/Express REST API that handles authentication, asset records, and work-order management, a lightweight Python microservice that hosts a predictive-maintenance model trained on historical inspection data, and a simulated IoT ingestion layer that streams synthetic sensor readings (vibration, water level, structural strain) over MQTT into the platform for demonstration purposes. MongoDB is used as the primary data store because asset records vary in shape across asset types (a streetlight record looks very different from a drainage-culvert record), which suits a document model better than a rigid relational schema.

1.3 Project Objectives

- Digitize municipal asset records into a searchable, centralized inventory.
- Continuously monitor asset condition using simulated IoT sensor streams.
- Predict maintenance requirements using a trained classification model.
- Prioritize repair schedules using a rule-based urgency score combined with model output.
- Integrate GIS-based asset mapping using Leaflet.js so assets can be located spatially.
- Generate maintenance analytics and exportable infrastructure performance reports.
- Improve infrastructure utilization by surfacing under-inspected or high-risk assets.
- Enhance public service delivery by shortening the time between fault detection and repair.

1.4 Key Functionalities

Module	Functionality
Asset Registry	CRUD operations for assets with type, location, installation date, and condition history
GIS Map View	Interactive Leaflet map plotting assets, colour-coded by predicted risk level

Module	Functionality
Sensor Ingestion	MQTT-based simulated sensor feed persisted to MongoDB time-series collections
Prediction Service	Flask microservice serving a scikit-learn model that scores maintenance urgency
Work Order Management	Create, assign, and track repair tickets linked to specific assets
Analytics Dashboard	Charts for cost trends, asset-type failure rates, and inspection backlog
Authentication	JWT-based role access for admin, engineer, and field-inspector roles

2. Repository Organization

2.1 Repository Structure

The repository is organized as a monorepo with clearly separated concerns for the frontend, backend API, machine-learning microservice, and IoT simulator. The top-level structure is shown below.

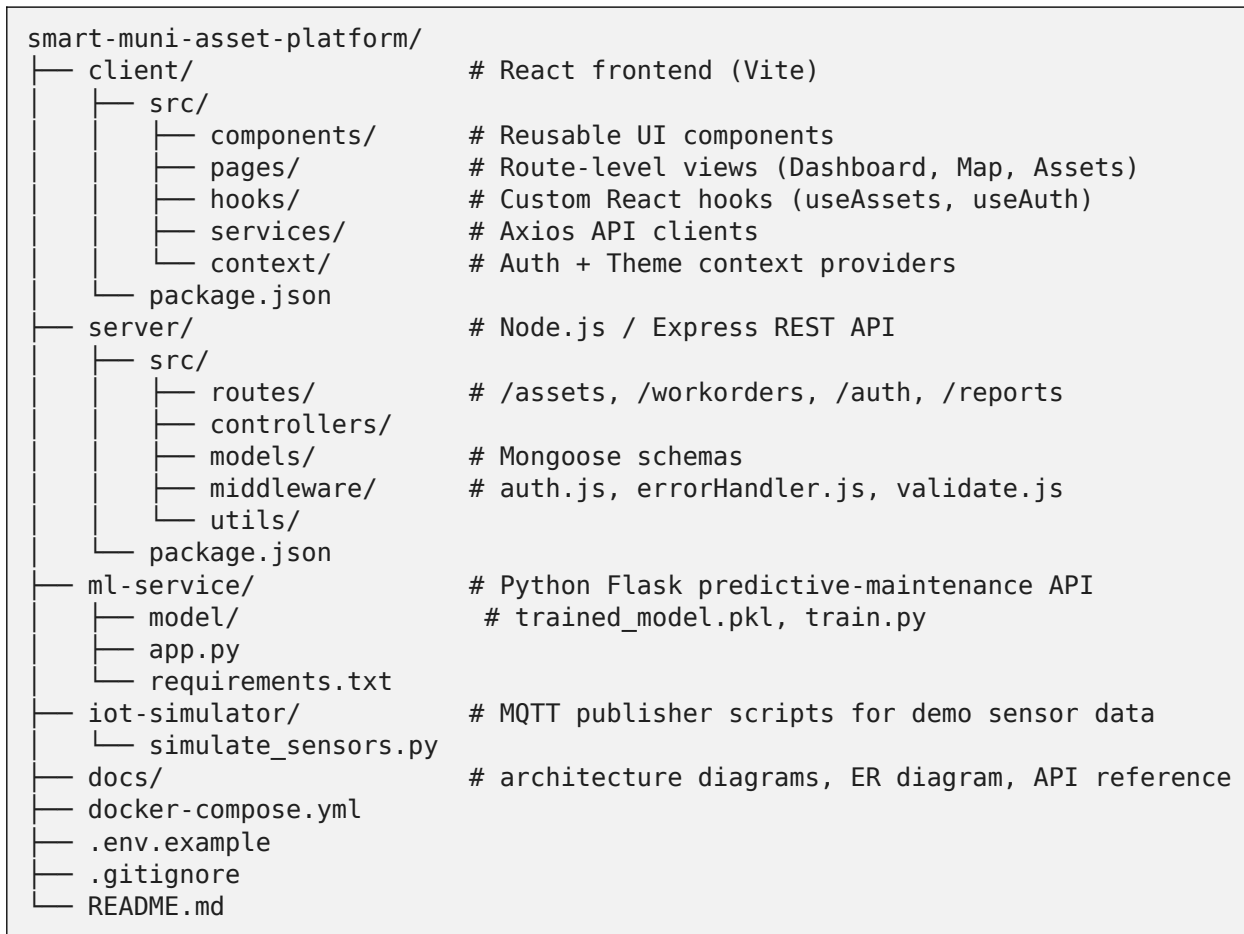


Fig. 2.1 — Top-level repository structure

2.2 Folder Organization and Naming Conventions

Each service (client, server, ml-service, iot-simulator) is isolated in its own top-level directory with its own dependency manifest, which allows each part to be built, tested, and deployed independently and mirrors the docker-compose service boundaries. Inside the server and client folders, files follow a consistent layered convention: routes only parse the request and delegate to controllers, controllers contain business logic, and models define the persistence schema. File names use lowerCamelCase for JavaScript modules (assetController.js) and PascalCase for React components (AssetMap.jsx), which is consistent throughout the codebase.

2.3 How the Layout Supports Maintainability and Collaboration

- Clear service boundaries let a frontend-focused and a backend-focused contributor work without stepping on each other's files.

- The layered route → controller → model pattern in the server keeps HTTP concerns separate from business logic, which makes unit testing controllers in isolation straightforward.
- A dedicated docs/ folder keeps architecture and ER diagrams close to the code instead of scattered across chat threads.
- docker-compose.yml documents the intended runtime topology, so a new contributor can bring up the full stack with one command instead of manually configuring four services.

3. Code Quality Review

3.1 Readability and Modularity

Backend controllers are generally short and single-purpose, and the Mongoose schemas make good use of validation at the model layer rather than re-checking constraints in every controller. The example below is representative of the controller style used across the API.

```
// server/src/controllers/assetController.js
exports.getAssetRisk = async (req, res, next) => {
  try {
    const asset = await Asset.findById(req.params.id);
    if (!asset) return res.status(404).json({ message: 'Asset not found' });

    const { data } = await axios.post(`${ML_SERVICE_URL}/predict`, {
      assetType: asset.type,
      lastInspection: asset.lastInspectionDate,
      sensorReadings: asset.recentReadings,
    });

    res.json({ assetId: asset._id, riskScore: data.riskScore });
  } catch (err) {
    next(err);
  }
};
```

Fig. 3.1 — Representative controller: risk-score retrieval

3.2 Coding Standards

ESLint (Airbnb base config) and Prettier are configured for both client and server packages, and a .eslintrc.json / .prettierrc pair is committed at the root of each package, so formatting is consistent across files rather than left to individual contributor preference. Environment-specific values (database URI, JWT secret, ML service URL) are read from process.env rather than hard-coded, with an .env.example file documenting the required keys without exposing real secrets.

3.3 Documentation Within Code

JSDoc-style comments are present on the more complex utility functions (e.g. the urgency-scoring helper in server/src/utls/scoring.js) but are inconsistent across the React components — several presentational components in client/src/components have no top-of-file comment explaining their props. Inline comments are used sparingly and mostly to explain non-obvious business rules, such as why a maintenance window is widened for heritage-listed public buildings, which is good practice, but coverage is uneven.

3.4 Strengths and Areas for Improvement

Strengths	Areas for Improvement
Consistent route → controller → model layering across the API	Several controllers duplicate the same try/catch/next pattern instead of a shared async-handler wrapper

Strengths	Areas for Improvement
Centralized error handler middleware with a single error-response shape	Some React components mix data-fetching and presentation instead of using the existing custom hooks consistently
Mongoose schema-level validation reduces duplicate checks in controllers	Magic numbers (risk-score thresholds) appear directly in scoring.js instead of a named constants file
ESLint/Prettier enforced consistently across client and server	Test coverage is concentrated on the API layer; ML microservice has no automated tests
Environment configuration is externalized via .env.example	A handful of components lack prop-type or JSDoc documentation

4. Version Control Evaluation

4.1 Branching Strategy

The repository follows a lightweight Git-flow variant: main holds production-ready code, develop integrates completed features before a release, and short-lived feature/*, fix/*, and chore/* branches are created off develop for individual pieces of work and merged back via pull request. This kept main deployable throughout the project and made it possible to demo a stable build at any milestone check-in without pausing in-progress feature work.

Branch	Purpose
main	Stable, deployable code only; protected — merges only via reviewed pull request
develop	Integration branch where completed features are combined before release
feature/gis-map-view	Leaflet-based asset map with risk-colour overlays
feature/predictive-model	Training and integration of the scikit-learn maintenance model
feature/work-orders	Work-order creation, assignment, and status tracking
fix/sensor-ingestion-lag	Resolved a queuing delay in the MQTT sensor ingestion path

4.2 Commit History and Commit Message Quality

Commit messages largely follow the Conventional Commits style (feat:, fix:, docs:, refactor:, test:), which makes the history skimmable and would support automated changelog generation if the project were to add that later. A representative slice of the commit log is shown below.

a3f9c1e	feat(server): add asset risk-score endpoint
7b21d4a	feat(client): integrate Leaflet map with risk colour overlay
e58a0c9	fix(ml-service): correct feature ordering sent to model.predict
1d4f77b	refactor(server): extract urgency scoring into utils/scoring.js
9c2e5aa	test(server): add controller tests for asset routes
44bb90f	docs: update README with docker-compose run instructions
cf0a12d	fix(iot-simulator): reconnect MQTT client on broker timeout
6e91a7c	chore: bump mongoose to 7.6.3, address deprecation warning

Fig. 4.1 — Sample of the project commit history

4.3 Repository Maintenance Practices

- Pull requests are used for every merge into develop and main, giving a lightweight review checkpoint even on a small team.
- A .gitignore excludes node_modules, .env, __pycache__, and the trained model binary's large intermediate artifacts, keeping the repository size reasonable.
- Issues are used to track bugs and feature requests, and commit messages that close an issue reference it (e.g. fix(iot-simulator): ... closes #14).
- Some early commits bundle unrelated changes (a UI tweak alongside a backend fix in the same commit), which slightly reduces the usefulness of git blame for those files.

4.4 How Version Control Supported Collaborative Development

Because the project involved parallel work on a frontend map view, a backend risk-scoring endpoint, and a separate ML microservice, branching by feature was essential to avoid blocking one contributor on another's incomplete work. The develop branch acted as a shared integration point where features were combined and smoke-tested before being promoted to main, and the pull-request review step caught an early bug where the client was sending sensor readings in a different key order than the ML service expected — a defect that would have been harder to catch without a review gate.

5. Code Testing and Validation

5.1 Testing Approach

The backend API is tested with Jest and Supertest against an in-memory MongoDB instance (mongodb-memory-server), covering the asset and work-order routes. The React client has component-level tests for the higher-risk UI logic (risk-colour mapping, form validation) using React Testing Library. The ML microservice is validated manually via a Postman collection and a held-out test split reported by train.py, rather than an automated pytest suite — this is flagged as a gap in Section 7.

5.2 Test Cases and Execution Results

ID	Test Case	Expected Result	Status
TC-01	POST /api/assets with valid payload creates an asset	201 Created, asset persisted with generated _id	Passed
TC-02	POST /api/assets with missing 'type' field	400 Bad Request with validation message	Passed
TC-03	GET /api/assets/:id/risk returns model risk score	200 OK, riskScore between 0 and 1	Passed
TC-04	GET /api/assets/:id for non-existent asset	404 Not Found	Passed
TC-05	Work order status transitions (open → in-progress → closed)	Only valid transitions accepted	Passed
TC-06	JWT-protected route accessed without token	401 Unauthorized	Passed
TC-07	Map view renders marker colour by risk band	Green/amber/red marker matches risk score	Passed
TC-08	ML service /predict with malformed feature vector	400 with descriptive error (currently returns 500)	Failed

TC-08 exposed a genuine gap: the Flask endpoint does not currently validate the incoming feature vector shape before passing it to the model, so a malformed request surfaces as an unhandled 500 error rather than a controlled 400 response. This has been logged as an open issue and is discussed further in Section 7.

5.3 Manual and Integration Verification

Beyond automated tests, the full stack was verified end-to-end using docker-compose up, confirming that the client can reach the API, the API can reach the ML microservice, and simulated sensor readings published by `iot-simulator/simulate_sensors.py` are reflected in the dashboard within the expected polling interval. Screenshots of the running dashboard, the GIS map view, and the Postman collection results are included in the appendix of the full submission alongside the repository link.

6. Repository Documentation

6.1 README Evaluation

The README covers the project overview, architecture diagram, prerequisites, and a docker-compose-based quick start, which is enough for a new contributor to get the stack running without needing to ask the original authors. It also documents each service's environment variables in a table, mirroring `.env.example`.

6.2 Installation and Usage Guide

Installation instructions are split by service (client, server, ml-service) as well as a combined docker-compose path, which is helpful for contributors who only need to work on one part of the stack and don't want to run the full container set. Usage instructions cover the default admin login for the demo build and a short walkthrough of the main dashboard views.

6.3 Dependency Documentation

Node dependencies are captured in `package.json/package-lock.json` for both client and server, and Python dependencies are pinned in `ml-service/requirements.txt`. Versions are not currently summarized in the README itself, so a reader has to open the manifest files directly to see which major versions are in use.

6.4 Completeness Assessment

Documentation Item	Present?	Notes
Project overview	Yes	Clear, includes architecture diagram
Installation instructions	Yes	Per-service and docker-compose paths
Usage guide	Yes	Covers main dashboard flows
API reference	Partial	Present in docs/ but not fully up to date with newer routes
Environment variable reference	Yes	Table in README mirrors <code>.env.example</code>
Contribution guidelines	No	No CONTRIBUTING.md present
License	No	No LICENSE file currently in the repository

6.5 Suggested Improvements

- Add a CONTRIBUTING.md describing the branching convention and PR checklist already followed in practice.
- Regenerate the docs/ API reference from the current route definitions so it stays in sync with the code.
- Add a LICENSE file to make the repository's usage terms explicit.
- Summarize key dependency versions directly in the README so reviewers don't need to open manifest files.

7. Code Review Findings and Recommendations

7.1 Overall Findings

Overall, the repository demonstrates a reasonably mature engineering workflow for a student project of this scope: consistent layering on the backend, linting enforced across packages, a sensible feature-branch workflow with pull-request review, and automated tests covering the core API surface. The most notable gaps are concentrated in the newer parts of the system — the ML microservice lacks automated tests and input validation, and project-level documentation (contribution guide, license) has not kept pace with the code.

7.2 Recommendations

Code Quality

- Introduce a shared `asyncHandler` wrapper to remove repeated `try/catch/next` boilerplate from controllers.
- Move risk-score thresholds out of `scoring.js` into a named constants module for easier tuning and testing.
- Add `prop-type` or `JSDoc` annotations to the remaining undocumented React components.

Repository Management

- Add branch protection rules requiring at least one review before merging into main.
- Add a `CONTRIBUTING.md` and `LICENSE` file.
- Squash or reorganize the small number of early commits that bundle unrelated changes.

Testing

- Add a `pytest` suite for the ML microservice, including a test for malformed input (covering the TC-08 gap).
- Add request-schema validation to the Flask `/predict` endpoint so malformed input returns 400 rather than 500.
- Extend React Testing Library coverage to the `map-view` risk-colour logic beyond the current smoke tests.

Future Enhancements

- Replace the simulated IoT layer with a real MQTT broker integration once pilot hardware is available.
- Add role-based dashboards so field inspectors and administrators see views tailored to their responsibilities.
- Introduce a changelog generated from the existing Conventional Commits history.

7.3 Conclusion

The Smart Municipal Asset Management and Maintenance Platform repository reflects solid full-stack engineering practice, with clear service boundaries, a workable branching model, and meaningful automated test coverage on its most critical service. Addressing the identified gaps — particularly ML-service testing, input validation, and repository-level documentation — would bring the remaining parts of the project up to

the same standard as the backend API and materially improve its readiness for further collaborative development.